MicroCRM : synthèse de la mise en place CI/CD

Plan de mise en place, environnements et correction du Dockerfile

1. Objectif et démarche

Automatiser la vérification, la construction et la livraison de MicroCRM (Angular 17 + Spring Boot 3.2, base HSQLDB en mémoire) avec GitHub Actions. Deux scénarios sont documentés : **A**, pipeline idéal (4 environnements déployés, e2e nocturnes, sauvegarde, rollback) et **B**, sous-ensemble réaliste aujourd'hui (pas d'hébergement, base en mémoire, un seul développeur). Le scénario B met en place **développement** (poste) et **test** (CI) ; pré-production et production viendront avec un hébergement.

2. Plan de mise en place

#	Étape	Résultat	État
1	Dépôt GitHub : branches **develop** et **main**, protection, CODEOWNERS	Modèle de branches features/* -> develop -> main	Branches créées ; protection à activer
2	Environnement de développement	Profil **dev**, lancement local	Fait
3	Environnement de test	Profil **test**, utilisé par la CI	Fait
4	Préparer le code : Karma headless, JaCoCo, ESLint, SonarQube Cloud	Mesures de qualité	Karma fait ; reste le reste
5	**ci.yml** (push, PR) : tests back et front, build, test de fumée Docker, job de synthèse	Contrôle obligatoire avant fusion	À écrire (exemple fourni)
6	Sécurité : CodeQL, Dependency Review, gitleaks, Dependabot, npm audit	Contrôles sur PR	À faire
7	**Dockerfile** corrigé et **compose.yml**	Images front et back fonctionnelles	Fait
8	**publish.yml** : build, test de fumée, scan Trivy, push GHCR, tags (SHA, branche, version), SBOM	Images publiées après fusion	À écrire
9	nightly.yml et weekly.yml : suite complète, audit, rebuild sans cache, scan	Détection des régressions	À écrire
10	Validation : features/test -> develop -> main	Parcours complet vérifié	À faire

3. Pipeline et seuils

Événement	Contrôles
Push sur features/* ou fix/*	Tests back et front, qualité, détection de secrets, build (retour rapide)
PR vers develop	Idem + sécurité et dépendances, build de l'image, test de fumée
PR vers main	Idem + scan des images (Trivy)
Fusion sur develop / main	Build, test de fumée, scan, puis publication GHCR (tags SHA immuable, branche, X.Y.Z-SNAPSHOT / vX.Y.Z)
Chaque nuit / chaque semaine	Suite complète sans cache, audit des dépendances / rebuild sans cache, scan, revue des tests instables

4. Environnements

Quatre environnements, chacun avec sa configuration. La **même image** est promue d'un environnement à l'autre : seule la configuration change.

Environnement	Où	Déclencheur	Profil / base	Rôle
Développement	Poste du développeur	Manuel	dev ; HSQLDB en mémoire, SQL et logs détaillés	Travailler sur une feature ou un fix ; aucun secret réel
Test	Runner GitHub Actions, puis instance sur develop	PR, fusion sur develop, nuit	test ; base jetable recréée à chaque exécution	Tous les tests ; jamais avec la config dev
Pré-production	Serveur (scénario A)	Fusion sur main	preprod ; données anonymisées	Réplique de la production, e2e nocturnes, restauration
Production	Serveur (scénario A)	Manuel + approbation	prod ; données réelles	Même image que la pré-production

Ce qui a été créé

- **Back** : `application-dev.properties` (main) et `application-test.properties` (test) ; `build.gradle` impose `SPRING_PROFILES_ACTIVE=test` à la tâche `test`, donc jamais le profil dev.
- **Front** : `src/environments/environment.ts` (dev), `environment.test.ts`, `environment.production.ts`, substitués par `fileReplacements` dans `angular.json` ; script `npm run test:ci` (Karma headless, couverture).
- **Compose** : `compose.yml` (services front et back), `compose.dev.yml` (profil dev), `compose.test.yml` (profil test + healthcheck du back).
- **Lancer** : `docker compose up --build` (dev par défaut) ; test : `docker compose -f compose.yml -f compose.test.yml up -d --build --wait`, puis `down -v`. Application sur https://localhost, API sur http://localhost:8080.

5. Correction du Dockerfile

Avant	Après	Raison
`node` et `gradle:jdk17` non épinglés	`node:20-alpine`, `gradle:8.7-jdk17-alpine`	Builds reproductibles ; Angular 17 exige Node 18.13+ / 20.9+
`alpine:3.19` + `apk add caddy`	`caddy:2.11.7-alpine`	Image officielle ; alpine 3.19 n'est plus supporté
`alpine:3.19` + JRE 21 (JDK 17 au build)	`eclipse-temurin:17-jre-alpine`	JRE seul, version alignée sur le build
`./gradlew build` (relance les tests)	`--no-daemon bootJar -x test`	Tests validés en CI ; pas de daemon dans un build jetable
Code copié avant les dépendances	`package*.json` + `npm ci`, puis code ; idem Gradle	Cache Docker efficace
`npx @angular/cli build`	`npm run build`	CLI figée par le package-lock.json
`EXPOSE 4200` sur le back	`EXPOSE 8080`	Port réel de Spring Boot
Exécution en root	Utilisateur `app` (front et back)	Réduire la surface d'attaque
Aucun HEALTHCHECK	Back : `wget` sur /organizations ; front : `nc -z localhost 80`	Savoir si le service répond ; base du `--wait` et du test de fumée
JAR `microcrm-0.0.1-SNAPSHOT.jar`	`microcrm.jar` (`archiveFileName`)	Le Dockerfile ne casse plus à chaque version ; la version vit dans le tag de l'image
Cible `standalone` + supervisord	Supprimée (et `supervisor.ini`)	Un processus par conteneur ; remplacée par Docker Compose

Caddy sert les fichiers statiques d'Angular (`file_server`), renvoie `index.html` pour les routes Angular (`try_files`), compresse en gzip et active le HTTPS automatique (certificat auto-signé en local, d'où https://localhost).

6. Reste à faire

- Écrire **ci.yml** par étapes : back-tests, front-tests, build, puis docker-smoke, sécurité, Sonar, et enfin la protection de branches (job de synthèse `ci-ok`).
- Valider `npm run test:ci` avec Chrome (non vérifié en local) ; ajouter JaCoCo, ESLint et la configuration SonarQube Cloud.
- Dockerfile : digest des images de base (Dependabot), `-XX:MaxRAMPercentage=75`, labels OCI.
- URL de l'API lue à l'exécution (aujourd'hui fixée à la compilation dans `environment.production.ts`) : condition pour promouvoir la même image entre environnements.
- Créer les profils **preprod** et **prod** et les environnements GitHub correspondants quand un hébergement existera.
- Hypothèses à confirmer : dépôt public, développeur seul, pas d'hébergement à court terme.